

Question 1

For the steps cost heuristic, the program iterates through each of the letters in the current state and counts how many letters are not correct for their position. This heuristic lower-bounds the unavoidable cost of words having to be in the dictionary. The minimum number of steps that you could take to get to the goal word is the number of letters that are wrong in the current state. The heuristic cannot overestimate the true optimal remaining cost since it will always return the minimum number of steps, whether or not it is actually possible.

For the Scrabble cost heuristic, the program finds the letters that need to be changed and adds the Scrabble replacement cost for each of the correct letters in those positions. Like the steps cost heuristic, this heuristic lower-bounds the unavoidable cost of words having to be in the dictionary and not every letter change forming a valid word. This heuristic cannot overestimate the true optimal remaining cost since, if possible, replacing the incorrect letters with the correct ones would be the minimal cost possibility.

For the frequency cost heuristic, the program returns 0 if it is at the goal. Otherwise, it returns 1 plus the rarity of the goal word. This heuristic ignores the constraints of having to change 1 letter at a time and words having to be included in the dictionary and assumes you will be able to go to the goal immediately. This heuristic cannot overestimate the true optimal remaining cost since it provides the minimum cost that is possible to take without constraints to reach the goal.

Question 2

Word Pair	Cost Measure	UCS Cost	UCS Nodes Expanded	A* Cost	A* Nodes Expanded	Reduction
why -> ask	steps	5	773	5	73	90.55627426
why -> ask	scrabble	9	765	9	13	98.30065359
why -> ask	frequency	38.85540652	808	38.85540652	688	14.85148515
bear -> duck	steps	4	691	4	6	99.1316932
bear -> duck	scrabble	11	2981	11	6	99.79872526
bear -> duck	frequency	38.2402284	1084	38.2402284	427	60.60885609

1. Yes, UCS and A* always returned the same optimal cost. Both of these searches are optimal, with A* search being optimal as long as the heuristic is admissible. Since the heuristics used are admissible and cannot overestimate the actual cost to the goal, A* returned the optimal cost.
2. For the why -> ask word pair, A* produced the largest percent reduction under the Scrabble cost measure. Similarly, for the bear -> duck word pair, A* also produced the largest percent reduction under the Scrabble cost measure. Since the heuristic gave the

cost to replace the wrong letters with the correct ones, it was able to give an informative heuristic and have the largest percent reduction.

3. For both word pairs, the frequency cost heuristic was the least informative since it expanded the most nodes. This makes sense since the heuristic I used only returns 2 values, one of them being 0 when the node is the goal node. Since the heuristic values for all nodes except for the goal node are the same, A* expanded more nodes.
4. Looking at the differences in expanded paths, it's evident that the steps and scrabble heuristics were more informative than the frequency heuristic. This is due to the fact that both of these heuristics provide the minimal possible cost to get to the goal, unlike the frequency heuristic, which provides one of 2 values.

Question 3

For the steps cost measure, yes, the optimal cost from W_1 to W_2 will always equal the optimal cost from W_2 to W_1 . The steps cost measure has already found the minimum steps required to get from W_1 to W_2 . Since actions are reversible and the cost to change any letter each time is consistently 1, the optimal cost to get from W_2 to W_1 will be the same steps, just reversed, therefore having the same cost.

For the Scrabble cost measure, no, the optimal cost from W_1 to W_2 will not always equal the optimal cost from W_2 to W_1 . Though actions are reversible, the cost of reversing an action using this cost measure is not the same. This is because when moving between states, the letter being inserted differs based on the direction; therefore, the cost also differs. For example, when moving between the words dog and cog, the action cost would be 3. When reversing this and going from cog to dog, the action cost would instead be 2. Because of this cost difference, the optimal costs will not always be the same.

Finally, for the frequency cost measure, no, the optimal cost from W_1 to W_2 will not always equal the optimal cost from W_2 to W_1 . The actions taken are reversible; however, the rarity costs between the 2 actions differ, making the optimal costs differ as well. For example, the action costs to go between the words "dog" and "cog" differ. The action cost to move between the words "dog" and "cog" would be about 12.39. However, when moving from the word "cog" to "dog", the action cost is about 6.82. Because of this action cost differential when reversing actions, the optimal cost from W_1 to W_2 will not always be the same for the reverse action.

Question 4

1. Model: Gemini 3.1 Pro

Prompt: In DOGCAT, the initial and goal states are lowercase words of the same length—three or four letters—and every legal state must appear in the supplied dictionary. A legal action changes exactly one letter. Under the frequency objective, entering word w costs $1 + \text{rarity}(w)$, where the nonnegative rarity value is stored in the dictionary.

The baseline heuristic $h_0(n)$ is the number of positions in which state n differs from the goal. Propose a rarity-aware heuristic intended to be more informative than h_0 while remaining admissible. Define it precisely and justify your proposal.

By “more informative,” we mean that the proposed heuristic should be at least as large as h_0 for every state and strictly larger for at least one state, without overestimating the true optimal remaining cost.

Proposed heuristic: Our rarity-aware heuristic adds the minimum rarity points you are **guaranteed** to pay, based on how far away you are:

- **0 letters away (At the goal):**
 - **Cost = 0**
 - **1 letter away:**
 - **Cost = 1 + (Goal word's rarity)**
 - **2 or more letters away:**
 - **Cost = (Letter differences) + (Goal word's rarity) + (Cheapest rarity among words connected to the goal)**
2. Pseudocode for heuristic

None

```
# Main Heuristic Function: h(n)
function Heuristic(current_word, goal_word, dictionary):
    # Case 1: At the goal state
    if current_word == goal_word:
        return 0

    # Calculate baseline distance h0
    h0 = HammingDistance(current_word, goal_word)
    goal_rarity = dictionary.getRarity(goal_word)

    # Case 2: Exactly 1 step away from goal
    if h0 == 1:
        return h0 + goal_rarity

    # Case 3: 2 or more steps away from goal
    else:
        min_neighbor_rarity = CalculateMinNeighborRarity(goal_word, dictionary)
        return h0 + goal_rarity + min_neighbor_rarity
```

3. This heuristic is admissible since it cannot overestimate the goal. If the current word is 1 letter away from the goal word, then the minimal cost to the goal would be 1 plus the rarity of the goal word; therefore, the heuristic does not overestimate in this case. For cases where the goal word is more than 1 letter away, the cost would be the number of steps to get there plus the goal word rarity and word rarities in between. Adding up the

number of steps to get to the goal, goal word rarity, and the minimal neighbor rarity is an admissible heuristic since it may not always include all the rarity costs, but it provides a lower bound of the actual cost to the goal and cannot overestimate.

4. When comparing the verified heuristic to the baseline heuristic, we find that the verified heuristic is more beneficial. Testing both heuristics for the frequency problems in Question 2, we find that each heuristic finds the same overall cost but differs in the number of nodes they expand, with the verified heuristic expanding fewer nodes. For example, for the why -> ask word pair, the baseline heuristic expanded 755 paths while the verified heuristic expanded 295 paths. This reduction is also consistent with the bear -> duck word pair, with the baseline heuristic expanding 720 paths while the verified heuristic expanded 180 paths. The proposed heuristic reducing the number of paths expanded, supports using the proposed heuristic over the baseline heuristic.